Journal of King Saud University – Computer and Information Sciences 34 (2022) 8658–8674

Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

Small files' problem in Hadoop: A systematic literature review

Raveena Aggarwal *, Jyoti Verma, Manvi Siwach

Computer Science Engineering, J.C. Bose University of Science and Technology, Faridabad, Haryana, India

ARTICLE INFO

Article history:
Received 27 April 2021
Revised 29 August 2021
Accepted 5 September 2021
Available online 15 September 2021

Keywords:
Hadoop
Hadoop Distributed File System
Small files
MapReduce
File merging

ABSTRACT

Apache Hadoop is an open-source software library which integrates a wide variety of software tools and utilities to facilitate the distributed batch processing of big data sets. Hadoop ecosystem comprises two major components - Hadoop Distributed File System (HDFS), which is primarily used for storage and MapReduce, which is primarily used for processing of the files. The performance of Hadoop holds back when it comes to storage and processing of small files. Small files are essentially the files that are significantly smaller in size when compared to the default block size of HDFS. This is because each small file consumes a block individually leading to excessive memory requirement, access time and processing time. Scaling the memory, allowing access latencies and processing delays beyond a limit is not an option. Henceforth, in this paper, a Systematic Literature Review has been performed to provide a comprehensive and exhaustive gestalt of the small files' problem in Hadoop. The paper defines a comprehensive taxonomy of Hadoop ecosystem and its' existent small files problem. Further, the study also attempts to critically analyze the solutions that have been proposed to overcome this problem. These solutions have been analytically studied to identify the set of parameters that should be considered in impending while proposing solutions pertaining to this problem.

© 2021 The Authors. Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

Contents

1. Introduction . . . 8659
   1.1. Motivation . . . 8659
   1.2. Our contributions . . . 8659
   1.3. Paper organization . . . 8659
2. Evolution of big data and computing paradigms . . . 8659
3. Related surveys . . . 8660
4. Research methodology . . . 8660
   4.1. Why Systematic Literature Review? . . . 8660
   4.2. Research questions . . . 8660
   4.3. Review protocol . . . 8661
      4.3.1. Search strategy . . . 8661
      4.3.2. Inclusion/Exclusion criteria . . . 8662
      4.3.3. Forward and backward citation . . . 8662
   4.4. Article selection . . . 8663
5. Taxonomy . . . 8663
   5.1. Hadoop ecosystem . . . 8663

* Corresponding author.
E-mail address: raveenaaggarwal@jcboseust.ac.in (R. Aggarwal).

Peer review under responsibility of King Saud University.

Production and hosting by Elsevier

https://doi.org/10.1016/j.jksuci.2021.09.007
1319-1578/© 2021 The Authors. Published by Elsevier B.V. on behalf of King Saud University.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

5.2. Factors leading to small Files' problem in Hadoop . . . 8666
5.2.1. High memory usage . . . 8666
5.2.2. Frequent communication among nodes . . . 8666
5.2.3. Processing overheads . . . 8666
5.2.4. Cross-node transmission of files . . . 8666
5.3. Techniques to overcome small files' problem in Hadoop . . . 8667
5.3.1. Metadata management optimization . . . 8667
5.3.2. Data management optimization . . . 8668
5.3.3. Miscellaneous . . . 8668
5.4. Performance indicators . . . 8669
6. Discussions . . . 8670
6.1. Comparison of various techniques used in literature . . . 8670
6.2. Comparison among top cited articles. . . . 8670
6.3. Cross – Analysis. . . . 8670
7. Future directions and gap areas identified . . . 8672
8. Conclusion . . . 8672
Declaration of Competing Interest . . . 8672
References . . . 8672

1. Introduction

With the unprecedented growth rate of data from multiple sources- be it social media, government organizations, businesses etc., it becomes a challenge to store and process the humungous data. But, data being one of the most valuable assets for any industry, it will be a callow act to loose on it. This brings in the demand for a distributed, cost-effective platform for big data storage and processing. Apache Hadoop's significant fault-tolerance, elevated reliability, high availability, and unlimited scalability, makes it an attractive choice for storing and processing huge amounts of datasets. Hadoop works on two major components - Hadoop Distributed File System (HDFS) and MapReduce. HDFS is a portable file system written in Java that can effortlessly host thousands of petabytes of data. MapReduce is a powerful programming tool that can process huge amounts of data in parallel.

A lot of applications generate small files in the form of log files, medical images, satellite images, meteorological data, PowerPoint presentations, etc. Big data is usually thought of as a collection of large files, overlooking upon the fact that a large number of small files also accounts to big data. This is exactly what happened with Hadoop. Hadoop was developed with intent to handle large datasets comprising of large files. So, its framework doesn't perform well when it comes to storing and processing huge number of small files. Addressing this challenge can majorly contribute in making Hadoop, an insuperable software solution in the field of big data and cloud computing. This paper focuses on the small files' problem in Hadoop.

1.1. Motivation

Small file handling is inevitable because data generated by several applications like social networking sites, e-business portals, e-learning applications, healthcare applications, financial firms, etc. are huge in figures, but significantly small in sizes, typically in the range of 10 KB to 10 MB. The current file systems, be it a local file system, a distributed file system or an object-based file system, show a smooth support for large sized files but most of them fail to serve the storage and processing of small files. This paper tries to apprehend the small files' problem existing in the current architecture adopted by one such file system - HDFS.

1.2. Our contributions

The major contributions of this paper are:

- A comprehensive taxonomy of small files' problem in Hadoop.
- Analysis of the various effects of storing large number of small files in the existing Hadoop architecture.
- A detailed analysis of the solutions adopted by Hadoop community and proposed by various researchers in the view of small files' problem.
- Identification of gap areas in the existing solutions that could be used for future reference by researchers for upcoming solutions.

1.3. Paper organization

The rest of this paper is organized as follows: Section 2 gives an overview of how the big data and computing paradigms evolved over a period of time. Section 3 enlists the related surveys present in the existing literature. Section 4 describes the detailed research methodology adopted while performing this Systematic Literature Review (SLR). Section 5 defines a novel comprehensive taxonomy for the said topic. Section 6 throws light on some major discussions – insights drawn from the entire review process. Section 7 discusses the gap areas that continue to exist in the small files' problem, leading to the future research directions. Section 8 finally concludes the article.

2. Evolution of big data and computing paradigms

Data is exploding – it is present everywhere. Fig. 1 illustrates the major milestones that appeared before reaching to the point of 'big' data. Although the concept of big data emerged quite recently, the act of gathering and analyzing data pioneered somewhere around 18th century when statistics began. Till late 19th century, all the data was collected and manipulated manually. In 1887, Herman Hollerith invented a computing system that could summarize the data stored on punch cards. This invention revolutionized the history of data computing. Thereafter, in 20th century, a lot of other inventions occurred in this field. In 1937, IBM got the contract to develop a punch card reading machine for US government to keep track of the contributions coming from the masses and the employers. Later, in 1943, British went one step ahead and developed a data processing machine called Colossus, with the capability to decipher Nazi codes during World War II. It reduced the data processing time from weeks to hours. This project largely represents the beginning of automatic data collection and processing. The concept of electronic data storage was conceived with the US government's proposal of building a data centre to store all tax returns and fingerprint records on magnetic tapes at a single location. In 1989, Tim Berners Lee invented World Wide Web, with a simple intention of data sharing across network. This

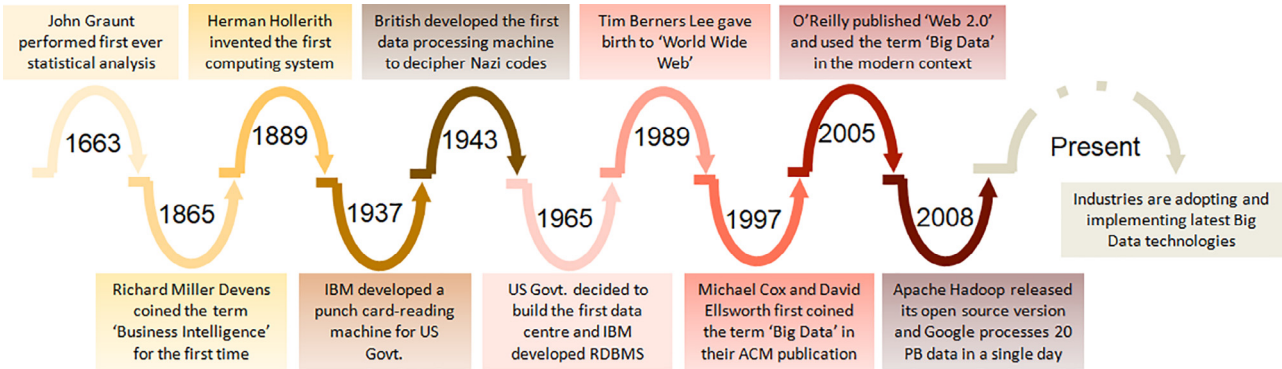

Fig. 1. Evolution of Big Data.

was where the things changed completely. Before the advent of internet, the growth rate of data was quite slow but it started to pace up expeditiously in 1990s.

Although Michael Cox and David Ellsworth first coined the term 'Big Data' in 1997 in their ACM publication, the modern context of big data is a legacy of 21st century. In 2001, Douglas Laney, who was working with Gartner (formerly known as Meta Group), introduced the 3 Vs of big data – volume, velocity and variety. In 2005, O'Reilly media started using big data to refer to a large amount of data that is not manageable with the traditional tools available to support the business intelligence. While all of this was happening around, Yahoo! released an open-source version of Hadoop under Apache Software Foundation (ASF). Hadoop is a widely accepted framework for storing and processing huge amounts of data in a scalable fashion. The developments in Hadoop have also come a long way. First being termed in 2006 as a sub-project to another project, it made its way to an independent and one of the most successful platforms for big data storage and processing. Fig. 2 (Aggarwal, 2019) illustrates how Apache Hadoop evolved.

3. Related surveys

In 2005, Doug Cutting at Yahoo! invented an open source implementation of Google File System (GFS) (Ghemawat et al., 2003) and MapReduce (Dean & Ghemawat, 2008) called as Apache Hadoop. Ever since this revolutionary invention, continuous research efforts are put in to identify any shortcomings and to develop strategies for them. This SLR is performed to discuss one such shortcoming of Hadoop – inefficiency in storing small files.

There are a few surveys performed in the past related to the small files' problem in Hadoop (listed in Table 1). This SLR differs from other reviews in many ways. First, it provides better and extensive publication coverage from 2010 to 2020. Second, it provides a comprehensive taxonomy and related publications. This will help the researchers to quickly map out the publications of their interest. Third, it performs a comprehensive analysis based on the techniques employed and the parameters considered for performance evaluation. Fourth, the main focus of the review is not only on the strategies adopted but also on the root causes and effects of small files, as well as, the performance of the suggested models/frameworks.

4. Research methodology

In order to structure the existing literature in the field of Hadoop's small files' problem, a systematic protocol has been followed, as depicted in Fig. 3. The key steps leading to the SLR are explained below:

- Identified the need of SLR as compared to other methodologies (refer section 4.1).
- Identified the research questions to justify the purpose of SLR (refer section 4.2).
- Identified a set of keywords used to query the prominent research databases and find publications from the top conferences/ workshops/symposia (refer section 4.3.1).
- Applied inclusion/ exclusion criteria on the retrieved set of papers, in order to extract a more refined set of papers that would better fit in the scope (refer section 4.3.2).
- Applied forward and backward citations to augment the set of relevant papers (refer section 4.3.3).
- Formulated an initial taxonomy and refined it while reviewing the obtained set of papers (refer section 5).
- Highlighted some important discussions derived from the existing literature in order to contribute to the research community working in the same/similar direction (refer section 6).
- Identified gaps, trends and future research directions based on the review (refer section 7).

4.1. Why Systematic Literature Review?

With the ever-increasing plethora of research being published in the field of big data, it is difficult to keep track of literature. The best solution to deal with the situation is to perform a review. The main aim of a review is to combine the individual studies into a single comprehensive overview in order to analyze the complex issues and resolve the discrepancies that may exist, keeping a topic in focus. This review can be a traditional narrative review, or a more contemporary SLR. The drawback of a narrative review is the possible presence of bias in selecting the articles to be reviewed (Uman, 2011), based on the availability and the line of author's conclusion. SLR, on the contrary, follows a pre-defined protocol for article selection and evaluation, which reduces the bias.

4.2. Research questions

Hadoop is efficient for storing and processing of small number of large files, rather than large number of small files. This is referred to as small files' problem. Large portion of the data being generated daily comprise of small files which contain valuable data and hence, becomes non-neglectable. Thus, the value of this SLR is that it focuses on helping the researchers to avail a comparison of wide-ranging solutions available in this domain in order to provide better solutions in future based on this study. Table 2 defines the research questions that are aimed for various reasons through this SLR.

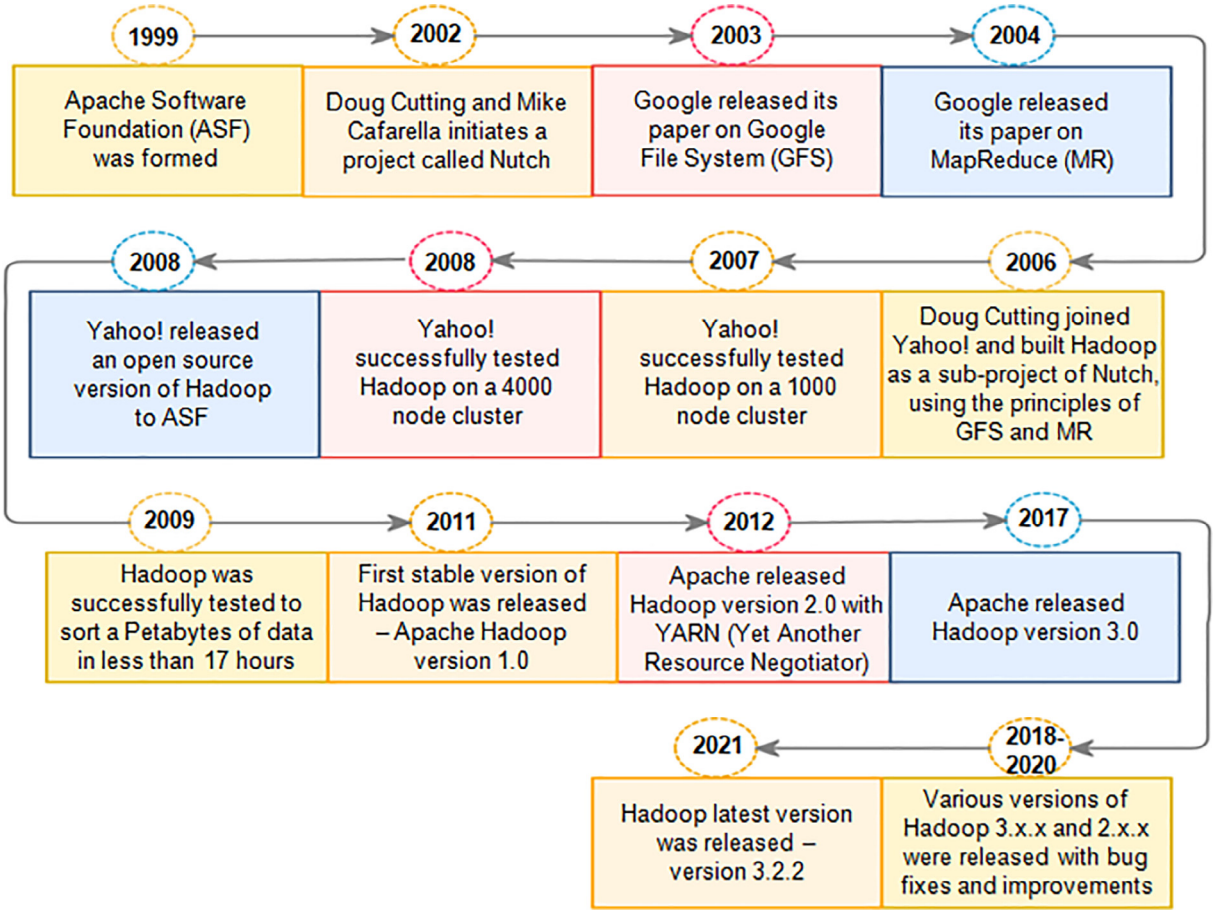

Fig. 2. Evolution of Hadoop.

Table 1
Related Surveys on Small Files' Problem.

Reference	Release Year	Coverage Period	Taxonomy	Comprehensive Analysis	Future Directions	Focus
(Masadeh et al., 2020)	2020	2012–2019	Not Available	Limited	Not Available	- Description of few native and proposed solutions
(Alange & Mathur, 2019)	2019	2009–2019	Not Available	Moderate	Not Available	- Comparative study of available techniques
(El-Sayed et al., 2019a,b)	2019	2009–2017	Not Available	Limited	Not Available	- Effect of small files in hadoop
(Huang et al., 2018)	2018	2009–2017	Not Available	Moderate	Moderate	- Effect of small files in hadoop - Detailed introduction of available techniques
(Rathidevi & Srinivasan, 2018)	2018	2008–2017	Not Available	Moderate	Limited	- Comparative study of available techniques
(Indira, 2017)	2017	2009–2016	Not Available	Moderate	Not Available	- Effect of small files in Hadoop

4.3. Review protocol

4.3.1. Search strategy

The main focus of this paper is to comprehend the architecture of Hadoop, and thereafter study the small files' problem in Hadoop. In order to achieve this, a large set of articles published in this field were retrieved from various sources. For the initial level, an extensive search of articles was performed on the following electronic research databases: Elsevier, IEEE Xplore, ACM digital library, Springer and Google Scholar. Apart from the aforementioned repositories, some other sources such as conference proceedings, workshops and symposia were also taken into consideration. The keywords were carefully identified and divided into four sets based on similarity. The first set of keywords revolved around the various adjectives used for a proposed strategy in research work. The second set of keywords comprised of words relating to the handling of small files. The third set itself contained the keywords for "small files". Finally, the fourth set contained the keywords relating to the platforms hosting small files' problem. Although the major focus in this article is on Hadoop platform, a lot of articles talk about small files' on cloud or in other distributed file systems. Hence, these keywords are included in this set in order to not lose on some major insights that can be drawn from such articles. A careful combinations of various identified keywords were used to

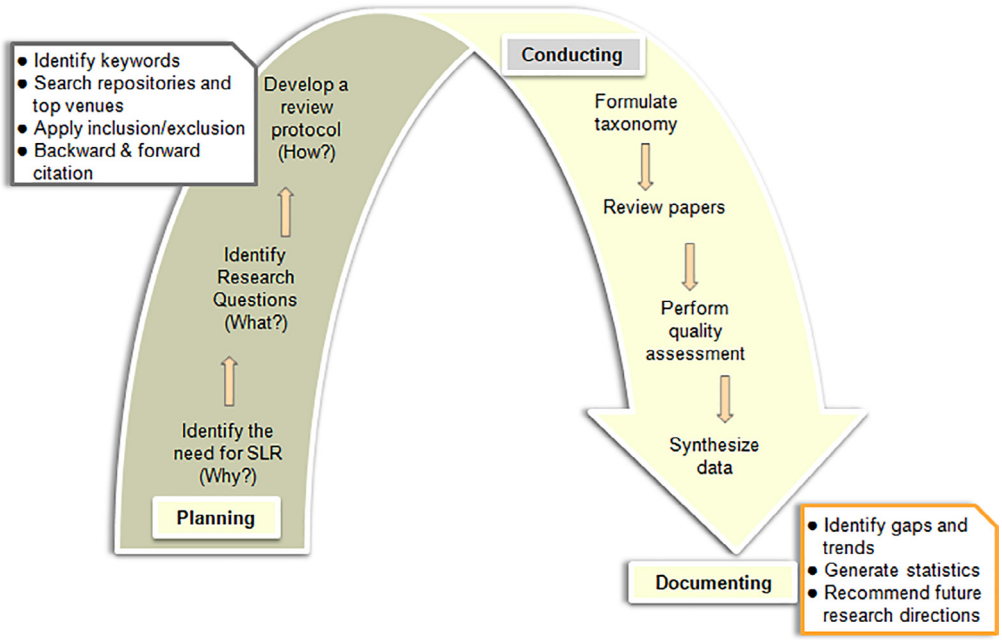

Fig. 3. Process of Systematic Literature Review.

Table 2
Research Questions identified.

ID	Research Question	Aim
RQ1	What are the various components of Hadoop and how do they contribute to small files' problem?	To build a comprehensive taxonomy of Hadoop and its small files' problem.
RQ2	What are the various strategies used to overcome the problem of small files in Hadoop?	To compare the different strategies and draw necessary conclusions based on them.
RQ3	What metrics should be used to evaluate the effectiveness of a proposed solution?	To evaluate the existing solutions and identify the possibilities of further enhancements.
RQ4	What are the gap areas that continue to exist in spite of the current research efforts?	To suggest an informed direction for future research in the field.

apply search queries on the above mentioned research databases. The keyword sets are defined below:

Keyword_set_1 = {optimization, improved performance, novel strategy}

Keyword_set_2 = {accessing, storing, handling, processing, prefetching}

Keyword_set_3 = {small file, small files, small data blocks}

Keyword_set_4 = {Hadoop, HDFS, MapReduce, cloud, Distributed File Systems}

Based on these sets, the following search query (Q) has been formulated:

Q = a OF b AND c IN d

where a ∈ Keyword_set_1, b ∈ Keyword_set_2, c ∈ Keyword_set_3 and d ∈ Keyword_set_4.

A sample query that can be formed using the above formula is: "Improved performance of accessing small files in Hadoop".

4.3.2. Inclusion/Exclusion criteria

A series of inclusion/exclusion criteria is applied on the set of research papers retrieved as a result of keyword-based search applied on the various databases mentioned in the above section. This is done to refine the result set according to the scope of this SLR, lest we shall divert from the main focus. The applied criteria are listed in Table 3.

4.3.3. Forward and backward citation

Once the set of papers retrieved from keyword-based search is refined by various inclusion/exclusion criteria, the search can be taken further with the help of forward and backward citations, also known as citation chaining or snowballing (Citation chaining in Google Scholar and PubMed, 2021). An article is linked to the articles it cites (backward citation) and to the articles that cite it (forward citation). In both the cases, the two articles are believed

Table 3
Inclusion/Exclusion Criteria.

Criteria	Inclusion	Exclusion
Publication Source	Journals, Conferences, Workshops and Symposia	Grey literature
Publication type	Systematic reviews, research articles, books, chapters	Meta-analysis, letters and news reports
Time frame	2010–2021	Before 2010
Language	English	Non-English
Review Type	Peer-Reviewed	Non Peer-Reviewed
Others	• High quality • Quantitative results	• Duplicates • Irrelevant content • Poor quality • Full-text not available

to discuss on the related topics. Backward citation helps in establishing links with the past publications with respect to the current article whereas the forward citation helps in establishing links with the articles that are published after the current article. Some relevant articles from the past can be accessed by looking at the references/bibliography section of the article. The credibility of an article can be assessed by looking at the number of articles that have cited it.

4.4. Article selection

The result set of publications retrieved after keyword-based search was huge in order. This set was refined through various procedures. A description of the same is depicted in Table 4. Initially, the articles were discarded based on the irrelevancy of titles. Next, the inclusion/exclusion criteria were applied to further narrow down the set of publications. As a result, 259 articles were carried forward for study. Out of these, 131 articles were discarded after reading their abstract. Remaining 128 papers were read in whole text, but 35 of these were found to be out of context or irrelevant to the topic. Finally, 93 articles were selected for detailed analysis.

Fig. 4 and Fig. 5 represents the percentage distribution of the selected articles based on their publishers and publication types, respectively. Almost 50% of the selected articles were published by IEEE, around 15% from Springer, followed by ACM and Elsevier. Rest of the articles were from the other publishers like Hindawi and Atlantis Press. Almost 54% of the selected articles are published in conferences, around 40% in Journals and the rest are from various workshops and symposia.

The keywords from the selected articles were analyzed for identifying the words with the most frequencies. Similar analysis was performed on the titles of the selected articles. The word clouds corresponding to the same are shown in Fig. 6(a) and Fig. 6(b).

5. Taxonomy

After careful examination of the selected articles, a novel taxonomy is constructed for comprehending small files' problem in Hadoop. The taxonomy first describes the Hadoop ecosystem by classifying it into five data layers namely storage, processing, querying, accessing and management. In reference to RQ1, studying through the ecosystem will further help us in analyzing the effects of storing large number of small files on various components/layers. The same has been summarized in Table 5. Thereafter, the factors leading to the small files' problem in Hadoop have been pointed out. Various metadata management optimization and data management optimization techniques have been adopted in the existing literature to overcome these factors. There exist certain other miscellaneous solutions as well. Finally, some parameters are identified to measure the performance of these strategies. Fig. 7 illustrates the proposed taxonomy.

5.1. Hadoop ecosystem

Each click on a device contributes to the voluminous generation of data. As the data on internet is increasing, so is the desire to be able to make data-driven-decisions in miniature time. This in turn triggers the need of a cost-effective platform to host the distributed storage technologies. Cloud computing is a one-stop solution to the above needs as it offers services remotely and considerably reduces the cost of distributed computing. The technical barriers standing in the way of cloud computing were majorly resolved by the advent of Apache Hadoop (Zhang et al., 2012a), an open source framework for distributed storage and processing (Alshammari et al., 2018).

Because of its capability to scale along the data, it is a widely accepted framework in a large section of industries. Yahoo!, Amazon, IBM, and Facebook are some of the prominent multinational technologies that deploy Hadoop to the best of their benefits (Dev & Patgiri, 2015). The major aspects of Hadoop ecosystem have been divided into five – data storage, data processing, data querying, data accessing and data management. There are several tools dedicated to each of the aspects. Major ones are discussed in the subsequent sections. Table 5 lists the effects of storing large number of small files in Hadoop on various components.

a) Data storage

Hadoop offers a light-weight distributed file system, known as Hadoop Distributed File System (HDFS) which is built on the principles of Google File System (GFS) (Ghemawat et al., 2003). It is an open source system designed to store humongous amounts of data. HDFS has a single master, several slaves' architecture. It mainly comprises of two types of nodes: A NameNode (which acts as a master) and multiple DataNodes (which acts as slaves). NameNode basically stores the reference to every file and their block locations in the cluster as a filesystem namespace. This namespace is referred as NameNode metadata. In order to maximize the efficiency, this metadata is placed in the main memory. NameNode is also responsible to regulate the file access operations. DataNodes are the nodes where the actual data resides. All the DataNodes may comprise different configurations as details of the underlying file system are abstracted (Peng et al., 2018).

HBase is an open-source, column-oriented, distributed, NoSQL database written in Java. It is built on the top of HDFS to provide distributed storage solutions. It is designed to perform operations with minimum latency and high speed updations with horizontal scalability. HBase, just like HDFS, is also built upon master–slave architecture.

A major shortcoming of this architecture is the lack of update operation i.e. a file can be only written once (Prasad et al., 2014). Also, NameNode acts as a single point of failure for this architecture. Another interesting fact to note is that a hadoop cluster can be easily scaled by simply adding extra nodes as per the requirements but, a single NameNode means that as soon as it's memory reaches a threshold value, the system cannot scale any further (Mukhopadhyay et al., 2014) because there will be no space left to maintain the metadata for any more files.

b) Data Processing

When we talk about big data, we are dealing with even the simplest of computations being performed on huge sets of data. In order to perform these computations well within acceptable time

Table 4
#Papers during article selection.

	Elsevier	IEEE	ACM	Springer	Others	Total Remaining
Keywords based results	2341	841	546	1345	57	5130
Excluded based on titles	1980	728	451	1167	11	793
Excluded applying criteria	253	82	57	131	14	256
Excluded after Abstract study	40	26	32	27	6	125
Excluded after detailed study	23	2	2	7	0	91

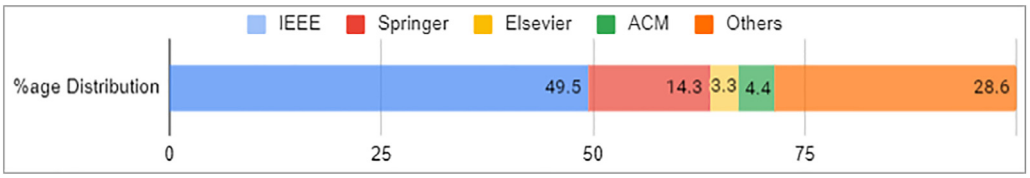

Fig. 4. Distribution of publications based on their publisher.

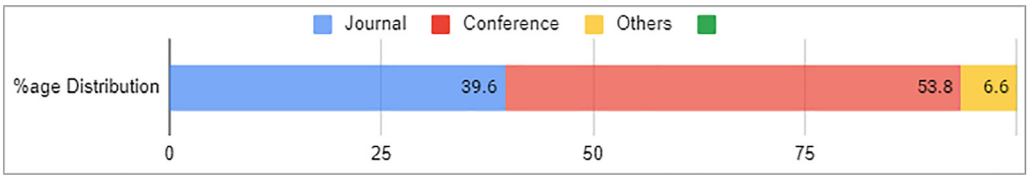

Fig. 5. Distribution of publications based on their type.

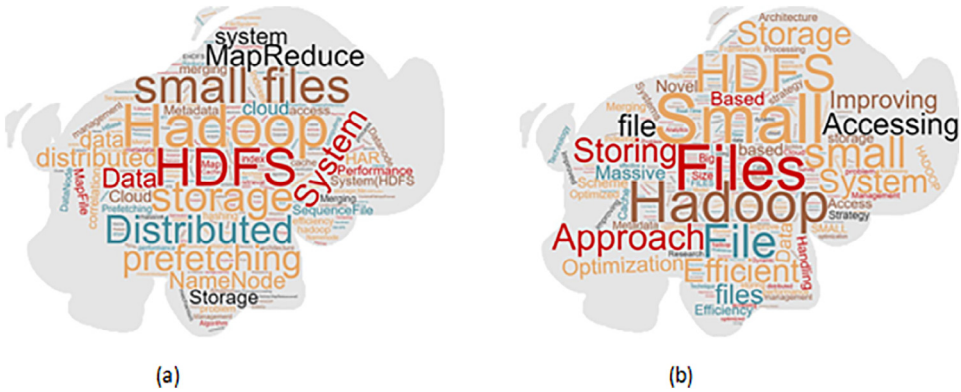

Fig. 6. (a) Word cloud of keywords of selected articles (b) Word cloud of titles of selected articles.

limits, we need to parallelize the computation by distributing among thousands of machines (Dean & Ghemawat, 2008). This became the fundamental idea behind the development of MapReduce. MapReduce is the Hadoop's fault-tolerant, reliable software framework written in Java, used for processing large files. Hadoop offers a streaming utility which validates a map/reduce code written in any language (Gohil & Panchal, 2014). The basic working principle of MapReduce is splitting a large/complex computation into smaller/simpler chunks (Mohandas & Thampi, 2011) and performing parallel computations over different machines. The MapReduce workflow is mainly divided into two phases- map phase and reduce phase, which are accompanied by an intermediate phase known as shuffle phase. The smallest unit of processing in MapReduce is known as InputSplit. Based on the total file size, the file to be processed is divided into InputSplits before the map phase, thereafter which, one map task is invoked on any of the slave nodes in the cluster corresponding to each InputSplit. All the processing is done in the form of < key, value > pairs. The mappers execute in parallel. Once their execution is over, the mapper output, also known as the intermediate output, is passed to shuffle phase where it is being merged and sorted based on the key values. In the next phase, this sorted output is processed by reduce task(s). The role of a reducer is to combine these sorted pairs of keys and values to produce the final output.

MapReduce was also earlier responsible for resource scheduling tasks but Hadoop 2.0 introduced YARN, Yet Another Resource Manager, which became the new in-charge of all the resource management, thereby controlling the tasks to be performed (Achandair et al., 2018). The motive behind development of YARN was to separate the processing layer from resource management layer.

Since an InputSplit in MapReduce comprise data from a single block, the small amount of data in a block will result in degrading the overall performance of the system. This is because the mappers are not able to perform well up to their maximum potential. Further processing overheads that occur in case of small files have been discussed in section 5.2.3.

c) Data Querying

The data stored on any system can only serve it right if there is an efficient mechanism to query that data with acceptable response time. Pig Latin (script-based) and HiveQL (Hive Query Language, SQL-like) are some of the query languages that can be used to fire queries on data stored on HDFS. They are capable of collaborating with MapReduce codes. Pig scripts are written in high level language (textual format). The compiler in Pig's infrastructure is built on top of MapReduce programs for performing faster analysis on large datasets in parallel fashion. Similar to MapReduce, pig also combines input from multiple small files in order to process more data in a single map and improve the efficiency. Hive is a cloud based data warehouse, built on the top of Hadoop system, to perform data analytics (R & G, 2017). Its working principle comprise of three primitive units – tables, partitions and buckets. The number of tables depends on the number of HDFS

Table 5
Various effects of storing large number of small files in Hadoop.

Device/ layer	Function	Effect	Source
HDFS Client	To retrieve the file system metadata from NameNode for each file access request	Frequent contact with NameNode, leading to its bottleneck performance	(Vorapongkitipun & Nupairoj, 2014)
NameNode	To store the file system metadata in main memory	Fast Consumption of main memory	(Vorapongkitipun & Nupairoj, 2014)
	To maintain a block report and update it regularly	Decreased throughout of the system	(Zhou et al., 2015)
DataNode	To store files as data blocks (smallest storage unit)	Low disk space utilization and storage space fragmentation	(Huang et al., 2018)
	To send the block reports to NameNode periodically	Increased network clogging and communication cost	(Dong et al., 2010a)
MapReduce	To generate input splits and assign a map tasks for each split, resulting in equivalent number of reduce tasks	Massive number of tasks created, leading to task scheduling overhead	(Prasad et al., 2014), (Zhou et al., 2015)
		Time consumption in start and shutdown of these tasks	(Huang et al., 2018)
		Wastage of cluster computing resources	(Huang et al., 2018)

directories, the tables are partitioned corresponding to the sub-directories within each directory, and the buckets represent files in each directory (table) or sub-directory (partition) (Costa et al., 2017; Thusoo et al., 2010). Hive is an excellent option for the applications that are working on query-based languages and looking for portability to Hadoop (Raj & D'Souza, 2019), however, the presence of large number of small files will result in excessive partitioning and a large number of buckets, degrading the system's performance.

Impala is the query engine used on top of HDFS. It is efficient in performing real-time analytics on big data (Siddiqa et al., 2017). Impala maintains a record of the HDFS metadata in its catalog server for improved access time. Instead of using Map-Reduce framework, it has its own in-memory processing engine. This helps Impala to operate the queries faster. But, the presence of large number of small files downturns the performance of this query engine, by slowing down the DDL (Data Definition Language) operations. It also consumes longer time and requires longer query plan (Naik & Gummalla, 2019).

d) Data Accessing

HDFS provides a steaming data access pattern. For efficient data processing, it follows the principle of Write-Once, Read-Many-Times (Korat & Pamu, 2012). The data access flow can be majorly divided into two phases – fetching the location of data and accessing the actual data (S. Fu et al., 2013). The former one consumes a lot more time as compared to latter. The client will first establish a connection with the NameNode and requests the file location of the desired file. NameNode will return all the locations of the files after authorizing the client. The client will then have to establish the connection with the nearest DataNode holding that file. Only then client will be able to sequentially access the file.

Hadoop incorporates various tools for easy access of data stored on HDFS. Apache Sqoop is a fault-tolerant command line interface used to transfer data in bulk. It needs a JDBC connector to establish connection and import data from various sources such as Oracle, MySQL, or any other database (Raj & D'Souza, 2019). Apache Spark can be used for scalable data streaming and parallelization. There

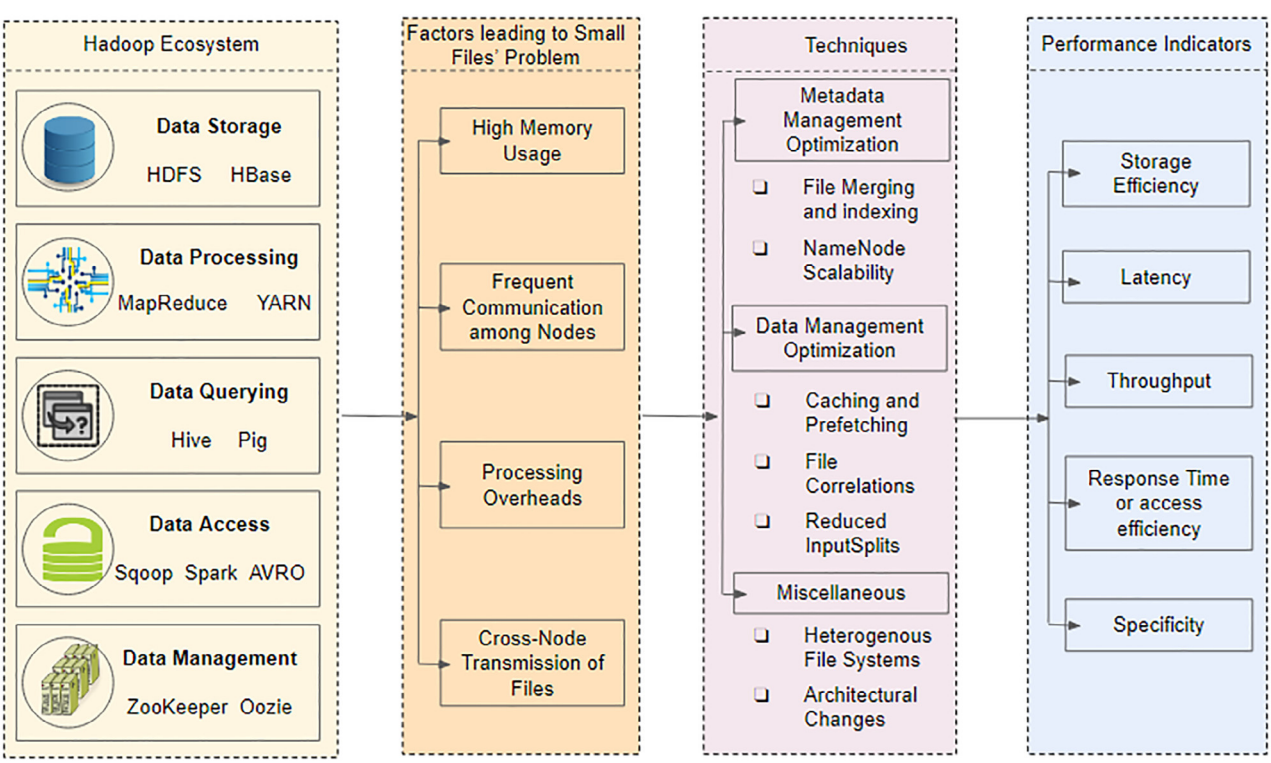

Fig. 7. Proposed Taxonomy for comprehending Small Files' Problem in Hadoop.

are other similar frameworks capable to operate in Hadoop environment such as Flume and Chukwa (Oussous et al., 2018). Whenever a file has to accessed, a Remote Procedural Call (RPC) has to be made to the NameNode (Peng et al., 2018). AVRO is used to make these RPC calls and further provide data serialization for easy transfer.

The protocol followed to access the files on HDFS are justified in case of large files as the cost of file operations can be amortized over the time spent in streaming data to/from the nodes (Niazi et al., 2018). However, in case of small files, the time to access the file exceeds the actual time spent on streaming/processing of the data held in it. Large number of small files will lead to frequent RPC calls, high network congestion and latencies.

e) Data Management

In order to maintain a proper coordination among various tools running simultaneously at different layers of Hadoop, it is important to monitor the workflow and provide desired management services. The data management layer offers tools for various functionalities such as data governance, data monitoring and job scheduling (Achandair et al., 2018). Data management in Hadoop is done at a centralized level. All the services need to be configured from time to time. The configuration and naming information are to be stored. The data stored on distributed systems needs to be synchronized every so often. The bugs or race conditions occurring while implementing tasks need to be handled. All this management is done by Zookeeper, a centralized service providing highly reliable distributed coordination (Apache ZooKeeper, 2021). Apart from these tasks, there is a need to schedule the Hadoop jobs in a sequential manner and ensure that they run in their proper sequence. This scheduling is performed by Apache Oozie.

If a lot of small files are present in the system, it consumes additional time for synchronization. Further, small files occupy the entire block in the system, leading to more blocks, and more tasks invoked. More tasks imply more scheduling, leading to excessive overhead.

5.2. Factors leading to small Files' problem in Hadoop

HDFS is designed mainly keeping in focus, the need to store and process huge datasets comprising of large sized files. The default size of a data block in an HDFS is usually larger i.e. n* 64 MB (n = 1, 2, 3...), as compared to any other file system. Any file which is smaller than this block size to a reasonable extent, is considered as a small file (Renner et al., 2017). With the increasing demand of internet of things, personalized recommendation systems, sensor based devices, and so forth, the data is generated in huge volumes but in small sizes. The inefficiency of Hadoop to store large number of small files is termed as small files' problem. This problem arises mainly due to the reasons discussed below. The number of research studies taking particular factors into consideration is depicted in Fig. 8.

5.2.1. High memory usage

While storing a large file in HDFS, the file is broken down to the size of HDFS blocks. The NameNode stores the metadata corresponding to each block in the DataNodes of the HDFS cluster. When we try to feed a large number of small files into this system, the performance degrades significantly. This is because the large files and the small files are treated indifferently i.e. the HDFS uses a separate block to store each file (Patel & Mehta, 2015), irrespective of its size. It means that a file of 50 MB stored on HDFS will consume the same amount of metadata space as consumed by a file size of 2 MB. So, in case of large number of small files, the size of metadata maintained by the NameNode outgrows the total of actual file sizes.

Apart from maintaining the namespace information regarding the files, the NameNode metadata also maintains a block report indicating the status of each block in the DataNodes. This block report is updated regularly by the means of heartbeat signal received by each block periodically. The update interval for block report is 21,600 s, by default (Zhou et al., 2015). During this update procedure, the system cannot accept any other operation to be performed on files, for the sake of synchronization. So, if we have extensive number of blocks, as observed in the case of small files, this procedure can take a few minutes and hence, degrade the throughput of the system. The above facets clearly indicate that the NameNode's memory utilization is a limiting factor for the performance and the capacity of cluster (Mukhopadhyay et al., 2014).

Furthermore, when we say that Hadoop treats the small files and large files indifferently, it also points to the fact that no matter the size of file, it will occupy a complete block of DataNode. This means that whether the file size is 10 KB, 2 MB or 10 MB, it will occupy a block of 64 MB (or 128 MB). This leads to the inefficient utilization of DataNode memory.

5.2.2. Frequent communication among nodes

Accessing a file on Hadoop requires a lot of communication. This is primarily due to the separation of data and its corresponding metadata across the file system (Matri et al., 2018). First, the client will need to establish a communication with the NameNode, seeking the file location. The NameNode will therefore look up the metadata and will return the requested location to the client. The client will next establish a communication with the desired DataNode, and will fetch the file accordingly. All the communications take place over the network, which may lead to network latencies. The high latencies occurring due to the access protocol of HDFS are not justified in the case of small files because the time spent in performing I/O operations exceed the time spent in the actual processing on the data (Ahad & Biswas, 2018b) and hence, there is a need to regulate the frequent communications among the clients, the metadata server and the DataNodes. The unavailability of prefetching and caching mechanisms in HDFS adds on to the I/O latency of the file system (Mao et al., 2015).

5.2.3. Processing overheads

Since we have a large number of small files, this means that processing these files will result in generating a large number of InputSplits (smallest unit of processing), one for each file. A mapper will be invoked corresponding to each InputSplit generated. Between each invoking, a context switch takes place. So, creating and closing so many processes (mappers) continuously and the corresponding context switches result in a lot of overhead. This situation can seriously hamper the performance of the system. Also, in case of processing a small file, the map phase has to deal with a single InputSplit (Zhou et al., 2015). In general case, the role of a reducer is to combine the outputs obtained from multiple mappers and reduce it to a simpler output. But, in this case, we have a single mapper which means that the reducer's output is already generated in the map phase. So, to extract the output from the map phase and feed that input for the reduce phase adds to another overhead here.

5.2.4. Cross-node transmission of files

In an effort to retrieve a large number of small files from the system, a lot of seeks occur, leading to a lot of hopping from one DataNode to another DataNode for each small file. This leads to excessive congestion on the network and reduced system throughput. The time spent in retrieving small files from the system cannot

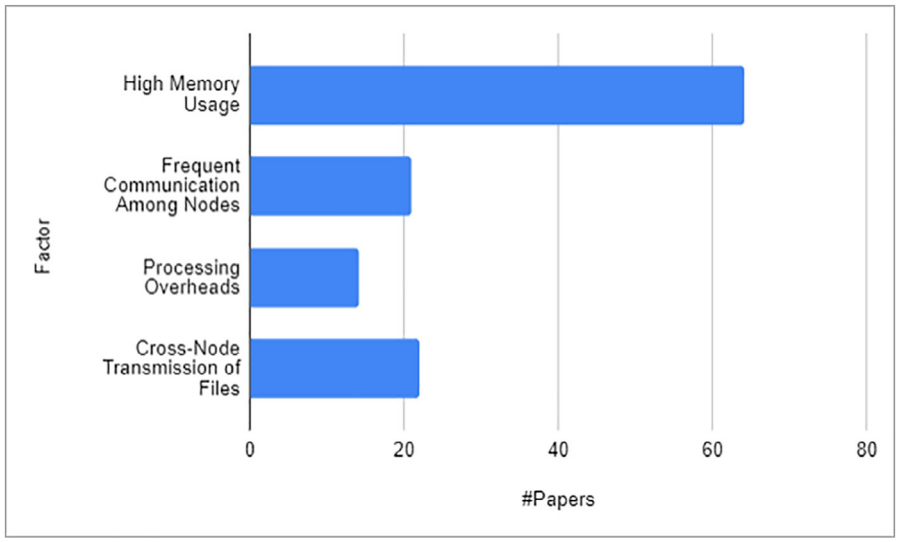

Fig. 8. Number of publications corresponding to the factors considered.

be amortized over the time spent in the actual processing of these files.

5.3. Techniques to overcome small files' problem in Hadoop

In the troupe of big data, a lot of small files are being generated. This drew significant attention towards the Hadoop's inefficiency in handling small files. As a result, a lot of techniques have been proposed in this direction. The available techniques have been studied and analyzed in an attempt to answer RQ2. As a result, these techniques are majorly divided into two broad categories: Metadata management optimization and Data management optimization. Other strategies have been listed under miscellaneous category. These are discussed in the sequent subsections. Table 6 lists the various strategies and the corresponding publications adopting them.

5.3.1. Metadata management optimization

Metadata optimizations maybe performed for any file sizes, but they mostly work well for small files. For larger files, they may rather lead to unsatisfactory performance (Kuhn et al., 2008). Various strategies have been adopted to optimize the metadata management in HDFS clusters for small files. These strategies are discussed below:

a) File merging and indexing

The simplest choice of techniques can be to reduce the memory footprint by merging the small files into larger ones and maintaining an efficient index corresponding to this merged file. Hadoop has itself offered some solutions for the same. For example, Hadoop Archives (HAR), SequenceFiles and MapFiles, file-crush utility, getmerge command (Ahad & Biswas, 2018b) and so forth. Using the 'archive' command in Hadoop, multiple small files can be packed into a single HAR file with indexing for easy access. However, these archives are immutable and the original files are not automatically deleted after the archive creation (C. Liu, 2016). SequenceFiles are the flat files stored in the form of key-value pairs where key holds the name of the file and value stores the data of the file. All the reading, writing and sorting operations performed on Hadoop are based on SequenceFile formats only (SequenceFile, 2021). But, SequenceFiles have low random read efficiency. MapFiles are similar to SequenceFiles except that they include a data file and an index file, sorted on index for allowing quick look up.

Many strategies have been developed to overcome the limitations of HAR, such as, Appendable Hadoop Archives (Renner et al., 2017), New Hadoop Archive (Vorapongkitipun & Nupairoj, 2014), HAR+ (Dev & Patgiri, 2015). The literature holds of several other solutions of this kind where certain new aspects have been used along file merging. For example – dynamic merging (Ahad & Biswas, 2018a), merging educational resource files (Dong et al., 2010a; Shen et al., 2010), merging based on file correlations (discussed in section 5.3.2 – b), considering the balance of merged data block (Cai et al., 2018; He et al., 2016), based on MapFile (Deeksha et al., 2017; Sheoran et al., 2017), based on the spatiotemporal parameters of the file (Xiong et al., 2019) and so forth. Also, while merging files, it is to be ensured that a merged file does not occupy more than one block, as it may reduce the access efficiency (Patel & Mehta, 2015).

The file merging process is mostly followed by an indexing scheme so that a particular file may be retrieved back easily with the help of indices. The existing literature is an evident of the efforts being put on the various efficient indexing schemes such as linear hashing (Tao et al., 2019), B + trees (Y. Wang et al., 2018), etc.

Most of the solutions observe that the process of merging files and indexing them can be a time consuming process. This implies that rather than improving the response time, this technique focuses on improving the system throughput (Zhang et al., 2012a).

b) NameNode scalability

A single NameNode helps in keeping the architectural configurations simpler but may lead to certain performance bottlenecks (Dev & Patgiri, 2016). This brings us to another way of dealing with the limitation of NameNode memory i.e. to use the concept of federated NameNodes (more than one node for storing metadata). This helps in dividing the load of a single NameNode over multiple machines. This strategy further helps in overcoming the single point of failure. In case of a single NameNode, if NameNode goes down, the entire system will go down. So, having a secondary NameNode can always rescue from such situations. However, this technique has been adopted by only quite a few researchers while dealing with small files (Aishwarya et al., 2014; Bi & Han, 2015; Dev & Patgiri, 2015; T. Gupta & Handa, 2016). This is because it

Table 6
Publications adopting various strategies.

Factor	Publications	#Papers
File merging and indexing	(Cui et al., 2012), (Renner et al., 2017), Bok et al., 2017b), (C. Liu, 2016), (Cai et al., 2018), (Dong et al., 2012), (B. Gupta et al., 2016), (Rathidevi & Parameswari, 2020), (Ahad & Biswas, 2018a), (Jian feng Peng et al., 2018), (Yan et al., 2014), (Dev & Patgiri, 2015), (Mohandas & Thampi, 2011), (Prasad et al., 2014), (Bisane et al., 2016), (Mao et al., 2015), (He et al., 2016), (Sheoran et al., 2017), (Korat & Pamu, 2012), (Meng et al., 2016), (Patel & Mehta, 2015), (Tao et al., 2019), (Zhou et al., 2015), (El-Sayed et al., 2019a,b), (Huo et al., 2016), (K. Wang et al., 2017), (Xiong et al., 2019), (Yang Zhang & Liu, 2012), (Zhao et al., 2012), (Siddiqui et al., 2020), (Ren et al., 2019), (B. Gupta et al., 2016), (X. Fu et al., 2016), (Khoulji et al., 2017), (Achandair et al., 2018), (Mohanty et al., 2018), (Sharma, 2018), (Y. Wang et al., 2018), (T. Gupta & Handa, 2016), (Jing & Tong, 2018), (Lyu et al., 2017), (Mu et al., 2016), (Cheng et al., 2017), (Chethan et al., 2010), (Deeksha et al., 2017), (Jayakar & Gurav, 2014), (Kumar, 2016), (Gohil et al., 2015), (Dong et al., 2010a), (Chandrasekar et al., 2013), (Gao et al., 2016), (Bok et al., 2017a), (T. Wang et al., 2015), (Li et al., 2013), (Aishwarya et al., 2014), (X. Liu et al., 2009), (Vorapongkitipun & Nupairoj, 2014), (Yang Zhang & Liu, 2012), (Yin Zhang et al., 2012b), (Nivedita & Geetha, 2017), (Ban & Jin, 2018)	61
NameNode scalability	(Mukhopadhyay et al., 2014), (Dev & Patgiri, 2015), (El Kafrawy et al., 2018), (Dev & Patgiri, 2016), (T. Gupta & Handa, 2016), (Aishwarya et al., 2014)	6
Caching and Prefetching	(Zhang et al., 2012a), (Mukhopadhyay et al., 2014), (Bok et al., 2017b), (Dong et al., 2012), (B. Gupta et al., 2016), (Jian feng Peng et al., 2018), (Yan et al., 2014), (Bisane et al., 2016), (Mao et al., 2015), (Meng et al., 2016), (Patel & Mehta, 2015), (J. Liu et al., 2010), (Huo et al., 2016), (K. Wang et al., 2017), (B. Gupta et al., 2016), (Dong et al., 2010b), (Khoulji et al., 2017), (Achandair et al., 2018), (T. Gupta & Handa, 2016), (Jing & Tong, 2018), (Lyu et al., 2017), (Cheng et al., 2017), (Deeksha et al., 2017), (Jayakar & Gurav, 2014), (Kumar, 2016), (Dong et al., 2010a), (Chandrasekar et al., 2013), (Gao et al., 2016), (Bok et al., 2017a), (T. Wang et al., 2015), (Aishwarya et al., 2014), (Yang Zhang & Liu, 2012), (Yin Zhang et al., 2012b), (Ban & Jin, 2018)	34
Reduced InputSplits	(Prasad et al., 2014), (Korat & Pamu, 2012), (Zhou et al., 2015), (Choi et al., 2018), (Phakade & Raut, 2014)	5
File correlations	(Cai et al., 2018), (Dong et al., 2012), (B. Gupta et al., 2016), (Rathidevi & Parameswari, 2020), (Jian feng Peng et al., 2018), (Bisane et al., 2016), (Mao et al., 2015), (Meng et al., 2016), (Patel & Mehta, 2015), (El-Sayed et al., 2019a,b), (Huo et al., 2016), (B. Gupta et al., 2016), (Dong et al., 2010b), (T. Gupta & Handa, 2016), (Jing & Tong, 2018), (Dong et al., 2010a), (Chandrasekar et al., 2013), (Gao et al., 2016), (T. Wang et al., 2015), (Nivedita & Geetha, 2017)	21
Heterogeneous file systems	(Niazi et al., 2018), (L. Zhang et al., 2017)	2
Architectural changes	(El Kafrawy et al., 2018), (J. Liu et al., 2010)	2

may solve the problem of small files' storage but gives rise to another set of research problems such as load balancing, increased complexity and synchronization among various NameNodes. Some authors have found an alternate to this – using a secondary storage media as an extension of NameNode memory (El Kafrawy et al., 2018).

5.3.2. Data management optimization

The blocks in Hadoop are of fixed size. As we know that each block stores a single file, so, if the file sizes are considerably smaller than the size of a block, it leads to inefficient disk optimization (El-Sayed et al., 2019a,b). Various strategies have been proposed by several researchers to optimize the data management in hadoop. These are discussed below:

a) Caching and prefetching

Cache is a scanty but faster memory used in the modern computers to store the frequently accessed files (Mukhopadhyay et al., 2014). The process of bringing the frequently accessed files from the slower main memory locations to this faster local memory is termed as 'prefetching'. Prefetching and Caching eliminates the need to initiate an RPC call for an already accessed file and reduces the overall access efficiency for the user. Some strategies propose a single cache, while others propose distributed caching schemes (Chen et al., 2012; Zhang et al., 2012b; J. Zhang et al., 2016; Bok et al., 2017b; Cheng et al., 2017; Bok et al., 2017a). In some articles, the frequently accessed metadata are prefetched while in others, the frequently accessed files are prefetched. In one of the articles, the files are prefetched based on their correlations (Dong et al., 2010b). Using such techniques helps in minimizing the disk I/O and the response time (V. Sharma & Barwar, 2020).

b) File correlations

File correlations and file merging are closely coupled together. Although file merging is a metadata optimization technique, but, to further improve the access efficiency of files merged together, many researchers have concluded that establishing file correlations comes as an aid. In the available literature, correlations have been established based on their access frequency (Meng et al., 2016), user access (Cai et al., 2018; Peng et al., 2018), upload time (Huo et al., 2016), read-write intensity of the files (X. Fu et al., 2016), file type/ file extensions (Kumar, 2016) and so forth.

c) Reduced InputSplits

InputSplit is the smallest unit of MapReduce processing. FileInputFormat is a module that creates a split per file, to be processed by a map task (Phakade & Raut, 2014). In case of large number of small files, a lot of InputSplits are generated, leading to the creation of excessive number of map tasks. MapReduce has offered a tool called CombineFileInputFormat which packs several InputSplits into one, based on the nodes and rack locality (Spivak & Nasonov, 2016). This ensures that each map task processes adequate amount of data from multiple blocks to enhance the MapReduce execution rate (Meng et al., 2016). One of the authors suggests that in case of small files, the MapReduce jobs shall be replaced by Map-only applications as the job of reduce phase has already been performed in the map phase (Zhou et al., 2015). In another study, overheads are further reduced by allowing multiple mappers to reuse the same JVM (Choi et al., 2018), cutting down the time and cost of creating the JVMs.

There are certain other strategies adopted for data management optimizations such as replication management, maintaining a balance among DataNodes, etc.

5.3.3. Miscellaneous

a) Heterogeneous file systems

One size doesn't fit all. Some files systems are designed to perform well on small files while some are designed to perform well on large files (L. Zhang et al., 2017). We can utilize the advantages of different distributed file systems and integrate them to

work together. HybridFS is a filesystem that employed this idea using three file systems - GlusterFS, HDFS, and Ceph (L. Zhang et al., 2017). On top of these file systems, a metadata management server is placed to perform three functions – intelligently place large and small files on different file systems, partially store the metadata of small files to improve the access efficiency and dynamically migrate files from one file system to another to maintain a balanced distribution of files among all.

b) Architectural changes

There are not many researchers who have followed this direction of making architectural changes in the Hadoop. In 2010, (J. Liu et al., 2010) suggested some changes in the architecture of HDFS to deal with the inefficient handling of small files. Typically, a data block can contain a single file and the metadata is only stored on the NameNode. According to the proposed solution, a block was allowed to store multiple small files. In consequence, the file-to-blocks mapping was replaced by block-to-files mapping. Also, the DataNodes were allowed to cache a part of metadata containing the small files stored in it. This helped in reducing the influence of a single master and reducing the read/write latencies. But, the overall solution was not optimal as the new structure will not allow a file to be split among multiple blocks, restricting the file size to the default block size.

Further, In 2017, (Eikafrawy et al., 2017) proposed another solution which also needed some architectural changes in HDFS. It made five changes at three different levels: (i) To reduce the memory usage – the NameNode will store the metadata only till a certain threshold value; after the threshold is reached, any further metadata is redirected to a secondary flash drive media; an indexer was introduced at NameNode to locate the metadata placed on the flash drive. (ii) To reduce the network flooding – each DataNode was allowed to hold a copy of the metadata of files contained in it, so that access requests for same can be handled without establishing connection with NameNode. (iii) To eliminate a single point of failure – the metadata of each DataNode was linked to the metadata of next and previous DataNode so that the access requests can be completely handled by the DataNodes if NameNode faces an issue. The experimental results for the above solution were analyzed later, by providing a simulation toolkit for HDFSx (El Kafrawy et al., 2018). HDFSx is however not a practical solution for handling small files.

Such strategies may lead to compatibility issues. Also, the modified systems need some specific environments to produce their best results (J. Zhang et al., 2016). Therefore, making architectural changes in Hadoop may not always be a good option.

c) Others

Some researchers have not opted for a solution based on any of the categories mentioned above. For instance, (Karan et al., 2019) simply proposed to store the metadata of small files in the secondary memory of the NameNode, rather than the primary memory. An AVL tree is maintained for faster access of metadata. Many of the merging strategies are applicable to only a specific type of application such as geospatial files (Cui et al., 2012; X. Liu et al., 2009), educational PPT files (Dong et al., 2010a), electronic pedigrees (Yin Zhang et al., 2012b), mp3 files (Zhao et al., 2012), multimedia files (Li et al., 2013) and so forth. Specificity is not introduced as an impact of a particular technique being implemented. Rather, specificity is more closely related to the solution being applicable to a particular file type or application. But, a solution is considered more efficient if it can be generalized to any file format or it can function under the minimum resource environment. Hence, lesser the specificity, better the solution.

5.4. Performance indicators

It is important to validate a proposed model by the means of its performance evaluation. The parameters shall be carefully and critically chosen for this purpose. Various strategies discussed in the literature employ one or more of the below mentioned parameters to indicate the performance of their proposed models. The parameters corresponding to the number of papers that have discussed them are depicted in Fig. 9.

- Storage Efficiency/ Cluster Scalability (SE/CS) – The primary motive of most of the proposed solutions seem to improve the storage efficiency and enhance the cluster scalability. This is mostly achieved by file merging strategies as the scalability of a cluster directly depends on the memory usage of NameNode.
- Pre-Processing Latency (PL) – Although most of the solutions talk about merging small files into larger ones, only quite a few seem to consider the latency that occurs due to the procedure involved in file merging. File merging may be an effective solution to deal with the memory constraints of NameNode but the time consumed in doing all the pre-processing work is a critical factor in analyzing the system performance.
- System Throughput (ST) – Throughput is the amount of work done by the system per unit time. In our context, it mainly deals with the time consumed in file upload/file writing operation.
- Response Time/ Access Efficiency (RT/AE) – According to a research, in most of the applications holding large number of small files, the 90% of user requests are made for accessing small files. However, the data accessed via these requests accounts for only about 10% of the entire data (Xiong et al., 2019). This is why response time or access efficiency is the most sought after criteria for performance evaluation after storage efficiency. Most of the researchers have talked about this parameter in their works. It deals with the time taken to access a file as opposed to the original Hadoop access protocol.

The above mentioned parameters shall be referred to in response of RQ3. Fig. 9 shows that the focus of the major articles referred was on improving the system storage efficiency or access efficiency (87.3% and 85.9% respectively). Quite a few articles con-

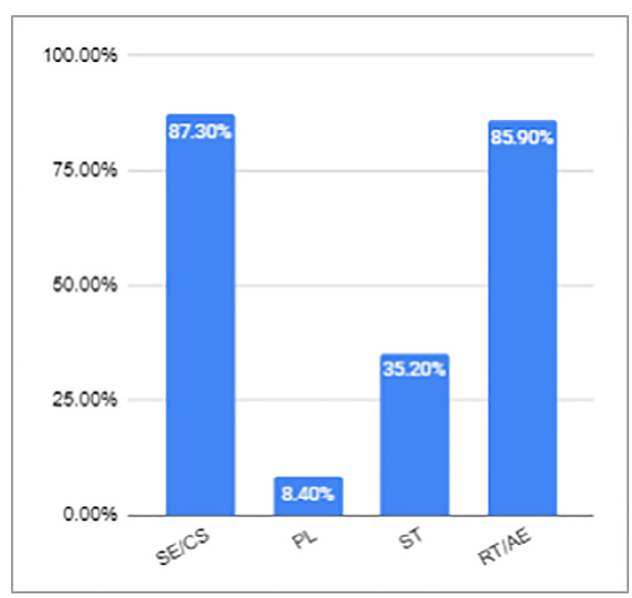

Fig. 9. %age distribution of Papers considering specific parameters for performance evaluation.

sidered the importance of achieving a wholesome of system throughput (35.2%). While very scanty number of articles considered the impact of pre-processing latency occurring while deploying various solutions (8.4%). The importance of these parameters is further explained in the next section by mapping it with various strategies discussed above.

6. Discussions

Many discussion areas arose while conducting this SLR. In order to address RQ2 and RQ3, some comparisons were drawn. These are discussed in the following subsections.

6.1. Comparison of various techniques used in literature

Table 2 mentions that RQ2 aims at comparing various strategies and draw necessary conclusions based on them. This and the next subsection follow to fulfill this aim by comparing the techniques proposed by various researchers in the existing literature. As described in the taxonomy, these techniques are divided into three categories. Table 7 enlists the advantages and shortcomings of the techniques proposed under each of these categories. It concludes that going for heterogeneous file systems or making architectural changes in the existing platform is a good option where the requirements are specific to the user or application. Also, it should be convenient to set up a given environment to bring out its best performance. But, for a general solution, it would be best to combine the techniques from metadata management optimization and the data management optimization. This will help to achieve maximum enhancement in the system.

6.2. Comparison among top cited articles

As this SLR covers a wide range of articles, only top cited articles have been considered for the comparison. Table 8 gives a comparison of these articles based on various parameters. Since there is no benchmark dataset available for the research in small files, the dataset considered in each article varies in size, number of files and file size distribution. Also, the experimental setup in each article shows a variety of cluster size and system configurations. This brings us to the inability of extracting statistical conclusions in comparison to various articles. Hence, the comparison is drawn on the basis of some common parameters of importance.

From Table 8, we can observe that although the block size is configurable in HDFS, the most commonly adopted is 64 MB. Also, the focus of major articles is to improve the storage efficiency, followed by the improvement in access efficiency. Very few articles paid attention to considering the enhancement in file write operations. Some of these articles offered solutions specific to an application, but mostly are general solutions. Further, we know that the merged files created in HDFS are immutable. This means that once created, no changes can be made - neither a new file can be added nor an existing file can be deleted or modified. However, this aspect was considered by a very few articles. Most of the solutions still produced immutable merged files. Another interesting aspect is related to the cut-off point. The cut-off point is a guiding point on how to separate large files from small files. The articles simply propose a solution to manage small files without concerning which files are actually small. Most articles do not even bring the cut-off point into discussion. A very few have considered a cut-off point but only theoretically, without any experimentation or practical reference. Only a certain article was found to provide a cut-off point on the grounds of experimentation (Dong et al., 2012). This should rather be a matter of great concern and hence, remains an open challenge for future researchers.

6.3. Cross – Analysis

A cross-analysis has been performed as an extended study of the taxonomy by mapping various techniques to the performance indicators. This study has been performed to support the clear value of performance indicators with respect to the different techniques. This analysis further supports the aim of RQ3 as it will help a researcher in deciding the parameters to consider while proposing a solution for small files' handling. Table 9 depicts the effect of adopting a certain technique on various aspects of system performance.

File merging improves storage efficiency while indexing provides an added support for improving the access efficiency of the merged files. This technique focuses on improving the system throughput. However, merging and indexing process leads to pre-processing latency and hence, reduces the overall response

Table 7
Comparison of various techniques used in literature.

Category	Technique	Advantages	Shortcomings
Metadata management optimization	File Merging and Indexing	• Contributes to storage efficiency • Improves namespace availability • Helps in enhancing system throughput	• Leads to pre-processing latency • Focuses on system throughput rather than response time • Source files are not deleted automatically • Merged files once created are mostly not appendable
	NameNode Scalability	• Removes single point of failure • Distributes NameNode load • Improves namespace availability • Better cluster scalability • Offers data isolation	• Increased complexity • May lead to load imbalance among NameNodes • Synchronization issues
Data management optimization	Caching and Prefetching	• Improved access efficiency • Disk I/O can be minimized • Avoids frequent RPC calls to NameNode • Minimizes communication among DataNodes	• Mostly implemented at client side • Implementation challenges in distributed environment
	File correlations Reduced InputSplits	• Improves access efficiency • Exploits MapReduce to its full potential	• Difficulty in establishing file correlations • Requires extra computation • Implementation challenges
Miscellaneous	Heterogeneous File Systems	• Utilize the advantages offered by different file systems to best of your requirement	• Trade-off among the capabilities of the chosen file systems • May lead to compatibility issues
	Architectural Changes	• Modify the available platform to the best of your requirement	• Compatibility issues • Need of specific environment to perform

Table 8
Comparison among top cited articles.

Paper	#Citations	Techniques Used	Dataset Description	HDFS Block Size	Storage Efficiency	Write Efficiency	Access Efficiency	Specific Solution?	Mutable Blocks?	Cut-off Considered?
(X. Liu et al., 2009)	277	1	558,726 files (3.6 GB); 96.19% files smaller than 16 KB	64 MB	Very High	Very High	Very High	Yes	No	No
(Dong et al., 2010a)	186	1,3,5	Multiple datasets; 98.25% files range between 32 KB and 256 KB	64 MB	Very high	Not considered	Very high	Yes	No	No
(Dong et al., 2012)	141	1,3,5	Dataset 1–119,600 files; 94% files less than 4.35 MB Dataset 2–42,043 files; 99% files less than 4.35 MB	64 MB	Very high	Not considered	Very high	No	No	Yes (4.35 MB)
(J. Liu et al., 2010)	99	3,7	200,000 files (4.2 GB); All files range between 4 KB and 1 MB	64 MB	Very high	Very High	Very high	No	Yes	No
(Chandrasekar et al., 2013)	91	1,3,5	100,000 files (3.2 GB); All files range between 10 KB and 150 KB	64 MB	Moderate	Very High	Moderate	No	No	No
(Yang Zhang & Liu, 2012)	61	1,3	10,000 files; All files are of less than 10 MB	64 MB	Very high	Not considered	High	No	No	No
(Vorapongkitipun & Nupairoj, 2014)	47	1	Four datasets – 20000, 40000, 60,000 and 80,000 files (1.4 GB, 2.8 GB, 4 GB and 5.6 GB respectively); All files range between 2 KB and 40 MB	128 MB	Very high	Not considered	Very high	No	Yes	No
(He et al., 2016)	39	1	4294 files (10.12 GB); 97.71% files lesser than 5 MB	64 Mb	High	High	Very high	Yes	No	No
(Mohandas & Thampi, 2011)	22	1	Experimental setup missing	64 MB	Results not produced	Results not produced	Results not produced	No	No	No
(Gohil et al., 2015)	20	1	20,000 files (1.1 GB); All files range between 10 KB and 126 MB	64 MB	Very High	Not considered	Not considered	No	No	Yes (80% of block size - only theoretical)
(Dong et al., 2010b)	20	3,5	100,000 files (305.38 GB); 78% files range between 256 KB and 8,192 KB	64 MB	Not Considered	Not Considered	Very High	No	No	No

Table 9
Mapping of various strategies to performance indicators.

	Storage Efficiency	Pre-processing Latency	System Throughput	Access Efficiency
File merging & Indexing	▲	▲	▲	▲
NameNode scalability	▲	⊗	▲	▲
Caching and Prefetching	⊗	⊗	⊗	▲
Reduced InputSplits	⊗	⊗	▲	⊗
File correlations	⊗	▲	▲	▲
Heterogeneous file systems	?	?	?	?
Architectural Changes	?	?	?	?

▲: Positive Increase, ▲: Negative Increase, ?: Cannot Say, ⊗: No Effect.

time. Implementing multiple namespaces contributes in improving storage efficiency, system throughput as well as access efficiency. It has never been linked to any kind of pre-processing latency in the existing literature. Caching and prefetching technique is not a sufficient solution when dealing with small files' problem because it improves access efficiency but has no effect on storage or throughput. So, it is better to use this technique in conjunction with some metadata optimization. Further, using file correlations while storing or merging small files can further help in improving the system throughput and access efficiency. Nevertheless, finding correlation among files can be a time consuming task, leading to latencies.

Besides, it can be observed from Table 9 that nothing can be said about how the heterogeneous file systems and architectural changes affect the system performance. This is because in the case of different file systems, the performance enhancements will totally depend upon the file systems chosen. Likewise, in case of architectural changes, the enhancements will depend upon the changes being made. Hence, these two techniques cannot be generalized as for other cases.

7. Future directions and gap areas identified

In the process of conducting the above SLR, RQ4 is answered by identifying the following gap areas that can be taken up by the researchers to further improve the handling of small files in Hadoop:

- There isn't a standard cut-off point available to distinguish between the large files and the small files. Different researchers have considered different values to decide whether a file is to be treated as small or not. (Mao et al., 2015) considered files smaller than 1 MB to be small, whereas (Niazi et al., 2018) considered the files larger than 64 KB as large files.
- Most of the solutions come with a trade-off between the metadata compression ratio and the framework performance. In future, some efforts may be invested to minimize this trade-off.
- Flash based devices maybe used to store the temporary files to reduce the memory footprint, considering the file lifetime.
- There exist some indexing schemes that allows to append new files to the existing archive/merged files but there is still no provision for update and delete operations (Huang et al., 2018).
- There should be a mechanism in the file systems to allow the users to tune the metadata and data optimizations according to their needs.

8. Conclusion

The paper focuses on presenting a broader view of the small files' problem that persists in Hadoop system. In view of the emerging importance of distributed platforms for big data storage such as Hadoop, there is a need of intensive studies and surveys in this field. Keeping this as a motivating factor, this paper tries to reflect upon some useful insights. The existing literature has been thoroughly reviewed to answer the four RQs that were introduced in the beginning of this paper. A proper methodology is adopted to perform this SLR which leads to some dominant findings. The paper first describes iteratively refined taxonomy which helps in understanding the Hadoop ecosystem. Based on this understanding, the drawbacks of storing large number of small files were investigated at various levels of the system. The most contemplated drawback is the memory limitation of NameNode. However, lots of small files further lead to the bottleneck performance at client side, low disk utilization at DataNodes, congestion over network, processing overheads, over/under-utilization of resources, data fetching overheads and so forth. The existing literature majorly examines the NameNode memory and the hence the solutions revolves around the same. The other factors mentioned above can be revisited and inspected in future to propose efficient solutions around them.

The studies further suggest that the small files' problem is a result of four major factors – 1) high memory usage at NameNode, 2) file access protocols leading to frequent communications among the nodes, 3) various processing overheads, and 4) the cross-node transmission of files. These factors are discussed in detail in Section 5.2. The literature is evident of the plentiful efforts made in this direction. Based on the comprehensive and comparative analysis of such efforts, this paper also identifies some parameters such as storage efficiency, pre-processing latency, system throughput, and access efficiency. These parameters can be used to assess a proposed solution. Along with improved memory efficiency, an optimal solution should simultaneously be able to improve the access efficiency of the files. Also, the solution should offer to improve the system throughput with minimum latencies. However, as discussed in Section 5.4, most of the existent solutions only attempts to improve the storage efficiency or cluster scalability to overcome the memory limitations.

In the process of investigating the above said parameters, the proposed strategies were classified into metadata management optimization, data management optimization and miscellaneous solutions. The solutions under the sub-categories of each of these categories are discussed in the paper. A meticulous study of these solutions helped the authors to draw a detailed comparison of various categories of strategies, followed by a comparison among the top cited articles from the selected articles. This comparison will further help the researchers to identify the best-fit category for their proposed solution. Furthermore, the paper is concluded by identifying the gap areas that still continue to exist in the path of overcoming the small files' problem. These gap areas include a dearth of articles that try to give a justified cut-off point to segregate small files from larger ones, existent trade-off between memory and performance and immutable merged files among others. The findings of this paper will encourage further research in the field.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Achandair, O., Bourekkadi, S., Elmahouti, E., Khoulji, S., & L. Kerkeb, M. (2018). solution for the future: small file management by optimizing Hadoop. International Journal of Engineering & Technology, 7(2.6), 221. doi: 10.14419/ijet.v7i2.6.10773.

Aggarwal, A. (2019, January). Hadoop | History or Evolution - GeeksforGeeks. https://www.geeksforgeeks.org/hadoop-history-or-evolution/.

Ahad, M.A., Biswas, R., 2018a. Dynamic Merging based Small File Storage (DM-SFS) Architecture for Efficiently Storing Small Size Files in Hadoop. Procedia Comput. Sci. 132, 1626–1635. https://doi.org/10.1016/j.procs.2018.05.128.

Ahad, M. A., & Biswas, R. (2018b). Handling small size files in hadoop: Challenges, opportunities, and review. In Advances in Intelligent Systems and Computing (Vol. 758). Springer Singapore. doi:10.1007/978-981-13-0514-6_62.

Aishwarya, K., Arvind Ram, A., Sreevatson, M.C., Babu, C., Prabavathy, B., 2014. Efficient prefetching technique for storage of heterogeneous small files in hadoop distributed file system federation. In: 2013 5th International Conference on Advanced Computing, ICoAC 2013. https://doi.org/10.1109/ICoAC.2013.6922006.

Alange, N., Mathur, A., 2019. Small Sized File Storage Problems in Hadoop Distributed File System. In: Proceedings of the 2nd International Conference on Smart Systems and Inventive Technology, ICSSIT 2019. Icssit. https://doi.org/10.1109/ICSSIT46314.2019.8987739.

Alshammari, H., Lee, J., Bajwa, H., 2018. H2hadoop: Improving hadoop performance using the metadata of related jobs. IEEE Trans. Cloud Comput. 6 (4), 1031–1040. https://doi.org/10.1109/TCC.2016.2535261.

Apache ZooKeeper. (n.d.). Retrieved April 17, 2021, from https://zookeeper.apache.org/.

Ban, Q., & Jin, Z. (2018). Strategy of storing and accessing small web log files on Hadoop. 2017 3rd IEEE International Conference on Computer and Communications, ICCC 2017. doi: 10.1109/CompComm.2017.8322739.

Bi, K., Han, D., 2015. Scalable Multiple NameNodes Hadoop Cloud Storage System. International Journal of Database Theory and Application 8 (1), 105–110. https://doi.org/10.14257/ijdta.2015.8.1.12.

Bisane, M.B., Prof, A., Chouragade, P.M., 2016. Improving Access Efficiency of Small Files in. 7 (2), 68–72.

Bok, K., Lim, J., Oh, H., Yoo, J., 2017a. An efficient cache management scheme for accessing small files in Distributed File Systems. In: 2017 IEEE International Conference on Big Data and Smart Computing, BigComp 2017. https://doi.org/10.1109/BIGCOMP.2017.7881731.

Bok, K., Oh, H., Lim, J., Pae, Y., Choi, H., Lee, B., Yoo, J., 2017b. An efficient distributed caching for accessing small files in HDFS. Cluster Computing 20 (4), 3579–3592. https://doi.org/10.1007/s10586-017-1147-2.

Cai, X., Chen, C., & Liang, Y. (2018). An optimization strategy of massive small files storage based on HDFS. 137(Jiaet), 225–230. doi: 10.2991/jiaet-18.2018.40.

Chandrasekar, S., Dakshinamurthy, R., Seshakumar, P.G., Prabavathy, B., Babu, C., 2013. A novel indexing scheme for efficient handling of small files in Hadoop Distributed File System. In: 2013 International Conference on Computer Communication and Informatics, ICCCI 2013. https://doi.org/10.1109/ICCCI.2013.6466147.

Chen, J., Wang, D., Fu, L., Zhao, W., 2012. An improved small file processing method for HDFS. International Journal of Digital Content Technology and Its Applications. https://doi.org/10.4156/jdcta.vol6.issue20.32.

Cheng, W., Zhou, M., Tong, B., Zhu, J., 2017. Optimizing small file storage process of the HDFS which based on the indexing mechanism. In: 2017 2nd IEEE International Conference on Cloud Computing and Big Data Analysis, ICCCBDA 2017. https://doi.org/10.1109/ICCCBDA.2017.7951882.

Chethan, R., J, J. K. S. G. H., & N, P. M. C. (2010). A Selective Approach for Storing Small Files in Respective Blocks of Hadoop. 461–465.

Choi, C., Choi, C., Choi, J., Kim, P., 2018. Improved performance optimization for massive small files in cloud computing environment. Ann. Oper. Res. 265 (2), 305–317. https://doi.org/10.1007/s10479-016-2376-0.

Citation chaining in Google Scholar and PubMed: AHE 330 W16 3796 - INFO LITERACY IN HEA SCI. (n.d.). Retrieved March 25, 2021, from https://canvas.seattlecentral.edu/courses/1252744/pages/citation-chaining-in-google-scholar-and-pubmed#/.

Costa, E., Costa, C., Santos, M.Y., 2017. Efficient big data modelling and organization for hadoop hive-based data warehouses. Lecture Notes in Business Information Processing 299, 3–16. https://doi.org/10.1007/978-3-319-65930-5_1.

Cui, J., Zhang, Y., Li, C., Xing, C.X., 2012. A packaging approach for massive amounts of small geospatial files with HDFS. In: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), 7418 LNCS. https://doi.org/10.1007/978-3-642-32281-5_20.

Dean, J., Ghemawat, S., 2008. MapReduce: Simplified data processing on large clusters. Commun. ACM 51 (1), 107–113. https://doi.org/10.1145/1327452.1327492.

Deeksha, S.P., Kanagavalli, R., Kavitha, K.S., Kavitha, C., 2017. Efficient Resolution for the NameNode Memory Issue for the Access of Small Files in HDFS. International Research Journal of Engineering and Technology(IRJET) 4 (4), 3–6 https://www.irjet.net/archives/V4/i4/IRJET-V4I4773.pdf.

Dev, D., Patgiri, R., 2015. HAR+: Archive and metadata distribution! Why not both? In: 2015 International Conference on Computer Communication and Informatics, ICCCI 2015. https://doi.org/10.1109/ICCCI.2015.7218119.

Dev, D., Patgiri, R., 2016. Dr. Hadoop: an infinite scalable metadata management for Hadoop—How the baby elephant becomes immortal. Frontiers of Information Technology and Electronic Engineering 17 (1), 15–31. https://doi.org/10.1631/FITEE.1500015.

Dong, B., Qiu, J., Zheng, Q., Zhong, X., Li, J., Li, Y., 2010a. A novel approach to improving the efficiency of storing and accessing small files on hadoop: A case study by PowerPoint files. In: Proceedings - 2010 IEEE 7th International Conference on Services Computing, SCC 2010, pp. 65–72. https://doi.org/10.1109/SCC.2010.72.

Dong, B., Zheng, Q., Tian, F., Chao, K.M., Ma, R., Anane, R., 2012. An optimized approach for storing and accessing small files on cloud storage. Journal of Network and Computer Applications 35 (6), 1847–1862. https://doi.org/10.1016/j.jnca.2012.07.009.

Dong, B., Zhong, X., Zheng, Q., Jian, L., Liu, J., Qiu, J., Li, Y., 2010b. Correlation based file prefetching approach for Hadoop. In: Proceedings - 2nd IEEE International Conference on Cloud Computing Technology and Science, CloudCom 2010, pp. 41–48. https://doi.org/10.1109/CloudCom.2010.60.

Eikafrawy, P.M., Sauber, A.M., Hafez, M.M., 2017. HDFSX: Big data distributed file system with small files support. In: 2016 12th International Computer Engineering Conference, ICENCO 2016: Boundless Smart Societies, pp. 131–135. https://doi.org/10.1109/ICENCO.2016.7856457.

El-Sayed, T., Badawy, M., El-Sayed, A., 2019a. SFSAN Approach for Solving the Problem of Small Files in Hadoop. In: Proceedings - 2018 13th International Conference on Computer Engineering and Systems, ICCES 2018, pp. 135–138. https://doi.org/10.1109/ICCES.2018.8639479.

El-Sayed, T., Badawy, M., El-Sayed, A., 2019b. Impact of Small Files on Hadoop Performance: Literature Survey and Open Points. Menoufia Journal of Electronic Engineering Research 28 (1), 109–120. https://doi.org/10.21608/mjeer.2019.62728.

El Kafrawy, P.M., Sauber, A.M., Hafez, M.M., Shawish, A.F., 2018. HDFSx: An Enhanced Model to Handle Small Files in Hadoop with a Simulating Toolkit. In: 1st International Conference on Computer Applications and Information Security, ICCAIS 2018. https://doi.org/10.1109/CAIS.2018.8442036.

Fu, S., Huang, C., He, L., Chaudhary, N., Liao, X., Yang, S., Wang, X., Li, B., 2013. IFlatLFS: Performance optimization for accessing massive small files. In: 20th Annual International Conference on High Performance Computing, HiPC 2013, pp. 10–19. https://doi.org/10.1109/HiPC.2013.6799116.

Fu, X., Liu, W., Cang, Y., Gong, X., Deng, S., 2016. Optimized Data Replication for Small Files in Cloud Storage Systems. Mathematical Problems in Engineering 2016, 1–8. https://doi.org/10.1155/2016/4837894.

Gao, Z., Qin, Y., & Niu, K. (2016). An effective merge strategy based hierarchy for improving small file problem on HDFS. Proceedings of 2016 4th IEEE International Conference on Cloud Computing and Intelligence Systems, CCIS 2016. 10.1109/CCIS.2016.7790278

Ghemawat, S., Gobioff, H., Leung, S.-T., 2003. The google file system. Operating Systems Review (ACM) 37 (5), 29–43. https://doi.org/10.1145/1165389.945450.

Gohil, P., Panchal, B., 2014. Efficient Ways to Improve the Performance of HDFS for Small Files. Computer Engineering and Intelligent Systems 5 (1), 45–49 http://iiste.org/Journals/index.php/CEIS/article/view/10306.

Gohil, P., Panchal, B., & Dhobi, J. S. (2015). A novel approach to improve the performance of Hadoop in handling of small files. Proceedings of 2015 IEEE International Conference on Electrical, Computer and Communication Technologies, ICECCT 2015. 10.1109/ICECCT.2015.7226044

Gupta, B., Nath, R., Gopal, G., K, K., 2016. An Efficient Approach for Storing and Accessing Small Files with Big Data Technology. International Journal of Computer Applications 146 (1), 36–39. https://doi.org/10.5120/ijca2016910611.

Gupta, T., Handa, S.S., 2016. An extended HDFS with an AVATAR NODE to handle both small files and to eliminate single point of failure. International Conference on Soft Computing Techniques and Implementations, ICSCTI 2015, 67–71. https://doi.org/10.1109/ICSCTI.2015.7489606.

He, H., Du, Z., Zhang, W., Chen, A., 2016. Optimization strategy of Hadoop small file storage for big data in healthcare. Journal of Supercomputing 72 (10), 3696–3707. https://doi.org/10.1007/s11227-015-1462-4.

Huang, L., Liu, J., & Meng, W. (2018). A Review of Various Optimization Schemes of Small Files Storage on Hadoop. Chinese Control Conference, CCC, 2018-July, 4500–4506. 10.23919/ChiCC.2018.8483588

Huo, Y., Wang, Z., Zeng, X., Yang, Y., Li, W., Zhong, C., 2016. SFS: A massive small file processing middleware in Hadoop. In: 18th Asia-Pacific Network Operations and Management Symposium, APNOMS 2016: Management of Softwarized Infrastructure - Proceedings, pp. 3–6. https://doi.org/10.1109/APNOMS.2016.7737234.

A.K. Indira, P.D. B. (2017). A review on small files in Hadoop a novel approach to undestand small files problem in Hadoop International Journal of Innovative Technology and Research 5 (4).

Jayakar, K.P., Gurav, Y.B., 2014. Managing Small Size Files through Indexing in Extended Hadoop File System. International Journal of Advance Research in Computer Science and Management Studies 2 (8).

Jing, W., Tong, D., 2018. An optimized approach for storing small files on HDFS-based on dynamic queue. In: Proceedings - 2016 International Conference on Identification, Information and Knowledge in the Internet of Things, IIKI 2016, 2018-Janua, pp. 173–178. https://doi.org/10.1109/IIKI.2016.55.

Karan, A., Rautaray, S.S., Pandey, M., 2019. A proposed approach for improving hadoop performance for handling small files. Advances in Intelligent Systems and Computing 813, 311–319. https://doi.org/10.1007/978-981-13-1498-8_28.

Khoulji, S., Achandair, O., Elmahouti, M., Kerkeb, M., 2017. Improving Small File Management in Hadoop. Transactions on Machine Learning and Artificial Intelligence 5 (4). https://doi.org/10.14738/tmlai.54.3333.

Korat, V. G., & Pamu, K. S. (2012). Reduction of Data at Namenode in HDFS using harballing Technique. 1(4), 635–642.

Kuhn, M., Kunkel, J., & Ludwig, T. (2008). Directory-based metadata optimizations for small files in PVFS. Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), 5168 LNCS, 90–99. 10.1007/978-3-540-85451-7_11

Kumar, L.P., 2016. Optimization Scheme for Storing and Accessing Huge Number of Small Files on HADOOP Distributed File System. International Journal on Recent and Innovation Trends in Computing and Communication 4 (2), 315–319.

Li, J., Lin, K., Wang, J., 2013. Design of the mass multimedia files storage architecture based on Hadoop. In: Proceedings of the 8th International Conference on Computer Science and Education, ICCSE 2013. Iccse, pp. 801–804. https://doi.org/10.1109/ICCSE.2013.6554018.

Liu, C., 2016. An improved HDFS for small file. In: International Conference on Advanced Communication Technology, ICACT, 2016-March. https://doi.org/10.1109/ICACT.2016.7423438.

Liu, J., Bing, L., & Meina, S. (2010). The optimization of HDFS based on small files. Proceedings - 2010 3rd IEEE International Conference on Broadband Network and Multimedia Technology, IC-BNMT2010, 912–915. 10.1109/ICBNMT.2010.5705223

Liu, X., Han, J., Zhong, Y., Han, C., & He, X. (2009). Implementing WebGIS on Hadoop: A case study of improving small file I/O performance on HDFS. Proceedings - IEEE International Conference on Cluster Computing, ICCC, August 2014. 10.1109/CLUSTR.2009.5289196

Lyu, Y., Fan, X., & Liu, K. (2017). An Optimized Strategy for Small Files Storing and Accessing in HDFS. 2017 IEEE International Conference on Computational Science and Engineering (CSE) and IEEE International Conference on Embedded and Ubiquitous Computing (EUC), 1, 611–614. 10.1109/CSE-EUC.2017.112

Mao, Y., Jia, B., Min, W., & Wang, J. (2015). Optimization Scheme for Small Files Storage Based on Hadoop Distributed File System. International Journal of Database Theory and Application, 8(5), 241–254. 10.14257/ijdta.2015.8.5.21

Masadeh, M.B., Azmi, M.S., Ahmad, S.S.S., 2020. Available techniques in hadoop small file issue. International Journal of Electrical and Computer Engineering 10 (2), 2097–2101. https://doi.org/10.11591/ijece.v10i2.pp2097-2101.

Matri, P., Perez, M.S., Costan, A., Antoniu, G., 2018. TýrFS: Increasing small files access performance with dynamic metadata replication. In: Proceedings - 18th IEEE/ACM International Symposium on Cluster Cloud and Grid Computing, CCGRID 2018, pp. 452–461. https://doi.org/10.1109/CCGRID.2018.00072.

Meng, B., Guo, W.B., Fan, G.S., Qian, N.W., 2016. A novel approach for efficient accessing of small files in HDFS: TLB-MapFile. In: 2016 IEEE/ACIS 17th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing, SNPD 2016, pp. 681–686. https://doi.org/10.1109/SNPD.2016.7515978.

Mohandas, N., Thampi, S.M., 2011. Improving hadoop performance in handling small files. In: Communications in Computer and Information Science, 193 CCIS (PART 4). https://doi.org/10.1007/978-3-642-22726-4_20.

Mohanty, A., Ranjana, P., & Subramanian, D. V. (2018). Small files consolidation technique in hadoop cluster. International Journal of Simulation: Systems, Science and Technology, 19(6), 31.1–31.5. 10.5013/IJSSST.a.19.06.31

Mu, Q., Jia, Y., & Luo, B. (2016). The Optimization Scheme Research of Small Files Storage Based on HDFS. Proceedings - 2015 8th International Symposium on Computational Intelligence and Design, ISCID 2015, 1, 431–434. 10.1109/ISCID.2015.285

Mukhopadhyay, D., Agrawal, C., Maru, D., Yedale, P., & Gadekar, P. (2014). Addressing name node scalability issue in hadoop distributed file system using cache approach. Proceedings - 2014 13th International Conference on Information Technology, ICIT 2014, December, 321–326. 10.1109/ICIT.2014.18

Naik, S., & Gummalla, B. (2019, May 9). Small Files, Big Foils: Addressing the Associated Metadata and Application Challenges - Cloudera Blog. https://blog.cloudera.com/small-files-big-foils-addressing-the-associated-metadata-and-application-challenges/.

Niazi, S., Ronström, M., Haridi, S., Dowling, J., 2018. Size matters: Improving the performance of small files in Hadoop. In: Proceedings of the 19th International Middleware Conference, Middleware 2018, pp. 26–39. https://doi.org/10.1145/3274808.3274811.

Nivedita, V., & Geetha, J. (2017). Optimization of hadoop small file storage using priority model. RTEICT 2017 - 2nd IEEE International Conference on Recent Trends in Electronics, Information and Communication Technology, Proceedings, 2018-Janua, 1785–1789. 10.1109/RTEICT.2017.8256906.

Oussous, A., Benjelloun, F.Z., Ait Lahcen, A., Belfkih, S., 2018. Big Data technologies: A survey. Journal of King Saud University - Computer and Information Sciences 30 (4), 431–448. https://doi.org/10.1016/j.jksuci.2017.06.001.

Patel, A., Mehta, M.A., 2015. A novel approach for efficient handling of small files in HDFS. *Souvenir of the 2015 IEEE International Advance Computing Conference*. IACC 2015, 1258–1262. https://doi.org/10.1109/IADCC.2015.7154903.

Peng, J., Wei, W., Zhao, H., Dai, Q., 2018. In: Hadoop Massive Small File Merging Technology Based on Visiting Hot-Spot and Associated File Optimization. Springer International Publishing. https://doi.org/10.1007/978-3-030-00563-4.

Phakade, P., Raut, S., 2014. An Innovative Strategy for Improved processing of small files in hadoop. International Journal of Application or Innovation in Engineering & Management (IJAIEM) 3 (7), 278–280.

Prasad, G., Nagesh, H. R., & Deepthi, M. (2014). Improving the Performance of Processing for Small Files in Hadoop: A Case Study of Weather Data Analytic. International Journal of Computer Science and Information Technologies, 5(5), 6436–6439. http://ijcsit.com/docs/Volume 5/vol5issue05/ijcsit2014050594.pdf.

R, A., & G, S. (2017). A Bird's Eye View on Big Data Analytics. International Journal of Engineering and Technology, 9(3), 1701–1706. 10.21817/ijet/2017/v9i3/170903046.

Raj, A., D'Souza, R., 2019. A Review on Hadoop Eco System for Big Data. International Journal of Scientific Research in Computer Science, Engineering and Information Technology 5 (1), 343–348. https://doi.org/10.32628/cseit195172.

Rathidevi, R., Parameswari, R., 2020. CSFC: A New Centroid Based Clustering Method to Improve the Efficiency of Storing and Accessing Small Files in Hadoop. International Journal of Recent Technology and Engineering 8 (4S5), 122–127. https://doi.org/10.35940/ijrte.d1014.1284s519.

Rathidevi, R., Srinivasan, S., 2018. Small files problem in Hadoop -A Survey. International Journal of Pure and Applied Mathematics 119 (15 Special Issue B), 2833–2841.

Ren, X., Geng, X., Zhu, Y., 2019. An Algorithm of Merging Small Files in HDFS. In: 2019 2nd International Conference on Artificial Intelligence and Big Data, ICAIBD 2019, pp. 24–27. https://doi.org/10.1109/ICAIBD.2019.8836971.

Renner, T., Müller, J., Thamsen, L., Kao, O., 2017. Addressing Hadoop's small file problem with an appendable archive file format. In: ACM International Conference on Computing Frontiers 2017, CF 2017, pp. 367–372. https://doi.org/10.1145/3075564.3078888.

SequenceFile - HADOOP2 - Apache Software Foundation. (n.d.). Retrieved April 14, 2021, from https://cwiki.apache.org/confluence/display/HADOOP2/SequenceFile.

Sharma, P. S. (2018). A Novel Architecture to Efficient utilization of Hadoop Distributed File Systems for Small Files. International Journal for Research in Applied Science and Engineering Technology, 6(5), 1934–1941. 10.22214/ijraset.2018.5314

Sharma, V., & Barwar, N. (2020). Data Management Techniques in Hadoop Framework for Handling Small Files: A Survey (pp. 425–438). 10.1007/978-981-15-4936-6_48

Shen, C., Lu, W., Wu, J., Wei, B., 2010. A digital library architecture supporting massive small files and efficient replica maintenance. In: Proceedings of the ACM International Conference on Digital Libraries, p. 391. https://doi.org/10.1145/1816123.1816195.

Sheoran, S., Sethia, D., Saran, H., 2017. Optimized MapFile Based Storage of Small Files in Hadoop. In: Proceedings - 2017 17th IEEE/ACM International Symposium on Cluster Cloud and Grid Computing, CCGRID 2017, pp. 906–912. https://doi.org/10.1109/CCGRID.2017.83.

Siddiqa, A., Karim, A., Gani, A., 2017. Big data storage technologies: a survey. Frontiers of Information Technology and Electronic Engineering 18 (8), 1040–1070. https://doi.org/10.1631/FITEE.1500441.

Siddiqui, I.F., Qureshi, N.M.F., Chowdhry, B.S., Uqaili, M.A., 2020. Pseudo-Cache-Based IoT Small Files Management Framework in HDFS Cluster. Wireless Pers. Commun. 113 (3), 1495–1522. https://doi.org/10.1007/s11277-020-07312-3.

Spivak, A., Nasonov, D., 2016. Data Preloading and Data Placement for MapReduce Performance Improving. Procedia Comput. Sci. 101, 379–387. https://doi.org/10.1016/j.procs.2016.11.044.

Tao, W., Zhai, Y., & Tchaye-Kondi, J. (2019). LHF: A new archive based approach to accelerate massive small files access performance in HDFS. Proceedings - 5th IEEE International Conference on Big Data Service and Applications, BigDataService 2019, Workshop on Big Data in Water Resources, Environment, and Hydraulic Engineering and Workshop on Medical, Healthcare, Using Big Data Technologies, 40–48. 10.1109/BigDataService.2019.00012

Thusoo, A., Sarma, J.S., Jain, N., Shao, Z., Chakka, P., Zhang, N., Antony, S., Liu, H., Murthy, R., 2010. Hive - A petabyte scale data warehouse using hadoop. Proceedings - International Conference on Data Engineering 996–1005. https://doi.org/10.1109/ICDE.2010.5447738.

Uman, L.S., 2011. Systematic reviews and meta-analyses. Journal of the Canadian Academy of Child and Adolescent Psychiatry. https://doi.org/10.30701/ijc.v39isuppl_b.856.

Vorapongkitipun, C., & Nupairoj, N. (2014). Improving performance of small-file accessing in Hadoop. 2014 11th Int. Joint Conf. on Computer Science and Software Engineering: "Human Factors in Computer Science and Software Engineering" - e-Science and High Performance Computing: EHPC, JCSSE 2014, 200–205. 10.1109/JCSSE.2014.6841867.

Wang, K., Yang, Y., Qiu, X., Gao, Z., 2017. MOSM: An approach for efficient storing massive small files on Hadoop. In: 2017 IEEE 2nd International Conference on Big Data Analysis, ICBDA 2017, pp. 397–401. https://doi.org/10.1109/ICBDA.2017.8078848.

Wang, T., Yao, S., Xu, Z., Xiong, L., Gu, X., & Yang, X. (2015). An effective strategy for improving small file problem in distributed file system. Proceedings - 2015 2nd International Conference on Information Science and Control Engineering, ICISCE 2015, 122–126. 10.1109/ICISCE.2015.35

Wang, Y., Li, Y.Q., Li, Y.W., Shi, Y., Li, W., 2018. Research of access optimization of small files on basis of B + tree on hadoop. Advances in Intelligent Systems and Computing 562, 197–204. https://doi.org/10.1007/978-981-10-4603-2_19.

Xiong, L., Zhong, Y., Liu, X., Yang, L., 2019. A Small File Merging Strategy for Spatiotemporal Data in Smart Health. IEEE Access 7 (1), 14799–14806. https://doi.org/10.1109/ACCESS.2019.2893882.

Yan, C., Li, T., Huang, Y., Gan, Y., 2014. Hmfs: Efficient support of small files processing over HDFS. In: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), 8631 LNCS(PART 2), pp. 54–67.

Zhang, J., Li, Q., Zhou, W., 2016. HDCache: A Distributed Cache System for Real-Time Cloud Services. Journal of Grid Computing 14 (3), 407–428. https://doi.org/10.1007/s10723-015-9360-9.

Zhang, J., Wu, G., Hu, X., Wu, X., 2012a. A distributed cache for Hadoop Distributed File System in real-time cloud services. Proceedings - IEEE/ACM International Workshop on Grid Computing 12–21. https://doi.org/10.1109/Grid.2012.17.

Zhang, L., Wu, Y., Xue, R., Hsu, T.C., Yang, H., Chung, Y.C., 2017. HybridFS-A High Performance and Balanced File System Framework with Multiple Distributed File Systems. Proceedings - International Computer Software and Applications Conference 1, 796–805. https://doi.org/10.1109/COMPSAC.2017.140.

Zhang, Y., Liu, D., 2012. Improving the efficiency of storing for small files in hdfs. In: Proceedings - 2012 International Conference on Computer Science and Service System, CSSS 2012, pp. 2239–2242. https://doi.org/10.1109/CSSS.2012.556.

Zhang, Y., Han, W., Wang, W., Lei, C., 2012b. Optimizing the storage of massive electronic pedigrees in HDFS. In: Proceedings of 2012 International Conference on the Internet of Things, IOT 2012, pp. 68–75. https://doi.org/10.1109/IOT.2012.6402306.

Zhao, X., Yang, Y., Sun, L.L., Huang, H., 2012. Metadata-aware small files storage architecture on Hadoop. In: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics): Vol. 7529 LNCS. https://doi.org/10.1007/978-3-642-33469-6_20.

Zhou, F., Pham, H., Yue, J., Zou, H., Yu, W., 2015. SFMapReduce: An optimized MapReduce framework for Small Files. In: Proceedings of the 2015 IEEE International Conference on Networking, Architecture and Storage, NAS 2015, pp. 23–32. https://doi.org/10.1109/NAS.2015.7255218.